DOM input event, which then propagates up to the form. The form has declared a Stimulus action input->search#submit so when it receives the input event, the submit method of this controller is invoked. All that method does is requestSubmit on the form, which submits the form and does so with the event Turbo Frames is looking for. (The Safari method for submitting forms doesn't send the event Turbo needs; it doesn't have this method, which is why we have to import the polyfill.)

And this works. Run it, type in the text box, and you'll get search results. Thanks to the Rails highlight helper, you'll even get the matching part of each string highlighted.

What's happening here is that we are using JavaScript to submit a regular form request, and then Turbo Frames is parsing the regular result from that HTTP call. Because the form tag specifies data-turbo-frame=search-results, Turbo is parsing out the search-results Turbo Frame from the HTML response and inserting it in the page where we have already placed a blank Turbo Frame with the search-results ID.

A few niceties are still worth adding. Right now there's no way to get rid of the modal once it pops up. Also, the search can get tangled up if the user types too fast. We can fix both of those problems with a little more Stimulus.

Adding JavaScript Actions to Search

What would be nice is for the modal to be reset when we delete all the characters in the search bar as well as when we click outside the window. However, the latter might be a bit tricky. To get the "outside the window" part to work, by definition we need to capture click events outside the scope of our Stimulus controller. Luckily, Stimulus has a way to do that.

First, we need to add an action to our search results object:

chapter_08/02/app/views/concerts/index.html.erb

```
<article
  class="fixed bg-gray-300 z-10
      rounded-3xl ring-4 ring-gray-800
```